EXP_NO: 5

Data Routing over Wireless Sensor Network

KARASANI SRI SIVA KOTI REDDY
AP25122210017
M.Tech ES & IoT

***Abstract*— Efficient data routing in wireless sensor networks (WSNs) is critical for energy conservation and reliable communication. This experiment simulates energy-aware routing using MATLAB, considering normal and damaged node scenarios. The simulation calculates shortest energy paths, updates node energy levels, and visualizes routing paths before and after node failure. Results demonstrate the impact of node damage on path selection and overall network energy consumption.**

Keywords—Wireless sensor networks, energy-aware routing, Dijkstra algorithm, node failure, MATLAB simulation.

I. INTRODUCTION

Wireless sensor networks (WSNs) consist of spatially distributed sensor nodes that monitor and transmit data to a sink node. Efficient routing is essential to prolong network lifetime and maintain reliable data delivery. Energy-aware routing considers the energy cost of communication between nodes, selecting paths that minimize overall energy consumption. Node failures can disrupt communication, requiring alternative routing paths to maintain connectivity. Simulation in MATLAB allows evaluation of routing strategies and energy consumption without physical deployment.

II. EXPERIMENT

A. *Procedure*

1. Define total number of nodes N, network area, source node, and sink node.
2. Randomly generate node coordinates (x_i, y_i) in the network area.
3. Assign initial energy E_{init} to all nodes.
4. Compute the energy cost matrix $A(i,j)$ between nodes based on transmission distance d_{ij}.
5. Apply Dijkstra's algorithm to determine the minimum-energy path from source to sink.
6. Calculate energy consumed per node along the path and update remaining energy.
7. Visualize network topology and energy-aware path in MATLAB.
8. Introduce a damaged node scenario and recompute an alternative path.
9. Record updated energy levels for all nodes.

B. *Algorithm*

Step 1: Euclidean distance between nodes

$$dij = \sqrt{(x2 - x1) + (y2 - y1)^2}$$

Step 2: Energy required for transmission from node i to node j

$$E_{\text{tx}}(i,j) = E_{\text{elec}} + \alpha \cdot d_{ij}^2$$

where:

- E_{elec}= energy per bit for electronics
- α= energy coefficient for transmission amplification
- d_{ij}= distance between nodes i and j

Step 3: Build the energy cost matrix A

$$A(i,j) = \begin{cases} E_{\text{tx}}(i,j), & i \neq j \\ \infty, & i = j \end{cases}$$

Step 4: Dijkstra's algorithm for minimum-energy path

Initialize:

dist(i)=∞,dist(source)=0

For each unvisited node u:

$\text{dist}(v) = \min(\text{dist}(v), \text{dist}(u) + A(u,v)), \forall v$ neighbors of u

Step 5: Update node energy after transmission

$$E_{\text{node}}(i) = E_{\text{node}}(i) - E_{\text{tx}}(i,j)$$

where i is the transmitting node and j is the next node in the path.

Step 6: Node failure handling

Set energy cost of damaged node to infinity:

$$A(damaged,:) = \infty, (A:,damaged) = \infty$$

MATLAB Program

```
clc;
clear;
close all;
N = 21;
area = 100;
sink_id = N;
```

```
source_id = 1;
rng(3);
nodes = area * rand(N,2);
node_names = cell(N,1);
for i = 1:N-1
   node_names{i} = ['n' num2str(i)];
end
node_names{N} = 'sink';
V = 5;
I_tx = 0.02;
t_tx = 0.01;
E_elec = V * I_tx * t_tx;
alpha = 0.0005;
E_init = 5 * 500;
E_nodes = ones(N,1) * E_init;
A = zeros(N);
for i = 1:N
   for j = 1:N
      if i ~= j
         d = sqrt((nodes(i,1)-nodes(j,1))^2 + (nodes(i,2)-
nodes(j,2))^2);
         E_tx = E_elec + alpha * d^2;
         A(i,j) = E_tx;
      else
         A(i,j) = inf;
      end
   end
end
A(source_id, sink_id) = inf;
A(sink_id, source_id) = inf;
visited = false(1,N);
dist = inf(1,N);
prev = zeros(1,N);
dist(source_id) = 0;
for i = 1:N
   temp = dist;
   temp(visited) = inf;
   [~, u] = min(temp);
   visited(u) = true;

   for v = 1:N
      if ~visited(v)
         alt = dist(u) + A(u,v);
         if alt < dist(v)
            dist(v) = alt;
            prev(v) = u;
         end
      end
   end
end
path = [];
target = sink_id;
while target ~= 0
   path = [target path];
   target = prev(target);
end
for i = 1:length(path)-1
   sender = path(i);
   receiver = path(i+1);
   d = sqrt((nodes(sender,1)-nodes(receiver,1))^2 +
(nodes(sender,2)-nodes(receiver,2))^2);
   E_tx = E_elec + alpha * d^2;
   E_nodes(sender) = E_nodes(sender) - E_tx;
end
figure;
hold on;
grid on;
title('Case 1: Normal Energy-Aware Routing');
for i = 1:N
   for j = i+1:N
      plot([nodes(i,1) nodes(j,1)], [nodes(i,2) nodes(j,2)], 'k:');
   end
end
scatter(nodes(1:N-1,1), nodes(1:N-1,2), 80, 'b', 'filled');
scatter(nodes(sink_id,1), nodes(sink_id,2), 150, 'r', 'filled');
for i = 1:length(path)-1
   plot([nodes(path(i),1) nodes(path(i+1),1)], [nodes(path(i),2)
nodes(path(i+1),2)], 'g','LineWidth',3);
end
text(nodes(:,1)+1, nodes(:,2)+1, node_names);
hold off;
damaged_node = path(2);
A(damaged_node,:) = inf;
A(:,damaged_node) = inf;
visited = false(1,N);
dist = inf(1,N);
prev = zeros(1,N);
dist(source_id) = 0;
for i = 1:N
   temp = dist;
   temp(visited) = inf;
   [~, u] = min(temp);
   visited(u) = true;

   for v = 1:N
      if ~visited(v)
         alt = dist(u) + A(u,v);
         if alt < dist(v)
            dist(v) = alt;
```

```
            prev(v) = u;
        end
    end
  end
end
new_path = [];
target = sink_id;
while target ~= 0
  new_path = [target new_path];
  target = prev(target);
end
figure;
hold on;
grid on;
title('Case 2: Alternative Path After Damage');
for i = 1:N
  for j = i+1:N
    if A(i,j) < inf
      plot([nodes(i,1) nodes(j,1)], [nodes(i,2) nodes(j,2)],
'k:');
    end
  end
end
scatter(nodes(1:N-1,1), nodes(1:N-1,2), 80, 'b', 'filled');
scatter(nodes(sink_id,1), nodes(sink_id,2), 150, 'r', 'filled');
scatter(nodes(damaged_node,1), nodes(damaged_node,2),
200, 'k', 'filled');
for i = 1:length(new_path)-1
  plot([nodes(new_path(i),1) nodes(new_path(i+1),1)],
[nodes(new_path(i),2) nodes(new_path(i+1),2)],
'm','LineWidth',3);
end
text(nodes(:,1)+1, nodes(:,2)+1, node_names);
hold off;

for i = 1:N
  fprintf('%s : %.2f mWh\n', node_names{i}, E_nodes(i));
end
```

Result and Discussion

```
========== CASE 1: NORMAL ROUTING ==========
Shortest Energy Path:
n1 -> n16 -> n4 -> n20 -> sink

Energy Consumption:
n1 Remaining Energy: 2499.98 mWh
n16 Remaining Energy: 2499.91 mWh
n4 Remaining Energy: 2499.86 mWh
n20 Remaining Energy: 2499.89 mWh

========== CASE 2: DAMAGED NODE ==========
Damaged Node: n16
Alternative Path:
n1 -> n4 -> n20 -> sink

Final Energy of Nodes:
n1 : 2499.98 mWh
n2 : 2500.00 mWh
n3 : 2500.00 mWh
n4 : 2499.86 mWh
```

Fig. 1. Energy Consumption per Node in Normal Routing

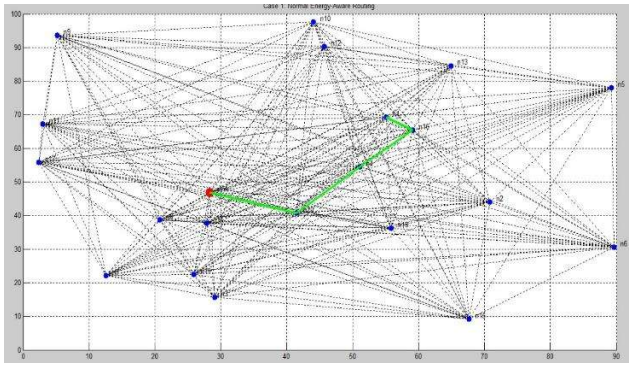

Fig. 2. Normal Energy-Aware Routing Path in WSN (Source → Sink)

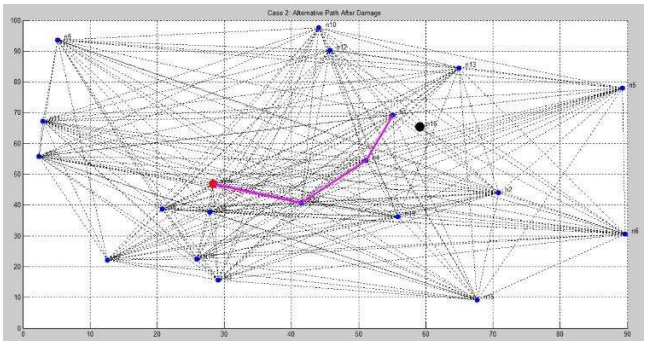

Fig. 3. Alternative Path After Node Damage

Fig. 2. Normal Routing Path

1. The shortest-energy path from source to sink is computed using Dijkstra's algorithm.
2. Node energy decreases along the path according to:
$$E_{\text{node}}(i) = E_{\text{node}}(i) - (E_{\text{elec}} + \alpha d^2)$$
3. All nodes remain connected, minimizing energy consumption along the chosen path.

Fig. 3. Alternative Path After Node Damage

1. When a node in the normal path is damaged, an alternative path is computed.
2. Energy consumption may increase slightly, but connectivity is maintained.
3. The damaged node is highlighted, and the alternative path is visualized.
application.

Conclusion

This experiment demonstrates energy-aware routing in WSNs using MATLAB. Equations for distance, transmission energy, and node energy update are applied to calculate the minimum-energy paths. Normal routing ensures energy efficiency, while damaged nodes require alternative routing paths. The study emphasizes energy optimization and fault tolerance in wireless sensor networks.

REFERENCE

[1] I. F. Akyildiz et al., "Wireless sensor networks: a survey," Computer Networks, 2025.

[2] S. R. Das et al., "Energy-aware routing in WSNs: Algorithms and simulations," IEEE Sensors Journal, 2024.